Chapter 8 ▪ Compiling and Scripting

This chapter introduces two techniques for processing code. You can use the compiler API when you want to compile Java code inside your application. The scripting API lets you invoke code in a scripting language such as JavaScript or Groovy.

8.1. The Compiler API

There are quite a few tools that need to compile Java code. Obviously, development environments and programs that teach Java programming are among them, as well as testing and build automation tools. Another example is the processing of Jakarta Server Pages—web pages with embedded Java statements.

8.1.1. Invoking the Compiler

It is very easy to invoke the compiler. Here is a sample call:

```
JavaCompiler compiler = ToolProvider.getSystemJavaCompiler();
OutputStream outStream = . . .;
OutputStream errStream = . . .;
int result = compiler.run(null, outStream, errStream,
    "-sourcepath", "src", "MyProg.java");
```

A `result` value of `0` indicates successful compilation.

The compiler sends its output and error messages to the provided streams. You can set these arguments to `null`, in which case `System.out` and `System.err` are used. The first argument of the `run` method is an input stream. As the compiler takes no console input, you can always leave it as `null`. (The `run` method is inherited from the generic `javax.tools.Tool` interface, which allows for tools that read input.)

The remaining arguments of the `run` method are the arguments that you would pass to `javac` if you invoked it on the command line. These can be options or file names.

8.1.2. Launching a Compilation Task

You can have more control over the compilation process with a `CompilationTask` object. This can be useful if you want to supply source from string, capture class files in memory, or process the error and warning messages.

To obtain a `CompilationTask`, start with a `compiler` object as in the preceding section. Then call

```
JavaCompiler.CompilationTask task = compiler.getTask(
    errorWriter, // Uses System.err if null
    fileManager, // Uses the standard file manager if null
    diagnosticListener, // Uses System.err if null
```

```
    options, // null if no options
    classes, // For annotation processing; null if none
    sources);
```

The last three arguments are `Iterable` instances. For example, a sequence of options might be specified as

```
Iterable<String> options = List.of("-d", "bin");
```

The `sources` parameter is an `Iterable` of `JavaFileObject` instances. If you want to compile disk files, get a `StandardJavaFileManager` and call its `getJavaFileObjects` method:

```
StandardJavaFileManager stdFileManager
    = compiler.getStandardFileManager(null, null, null);
Iterable<JavaFileObject> sources
    = stdFileManager.getJavaFileObjectsFromStrings(List.of("File1.java", "File2.java"));
JavaCompiler.CompilationTask task
    = compiler.getTask(null, null, null, options, null, sources);
```



Note: The `classes` parameter is only used for annotation processing. In that case, you also need to call `task.processors(annotationProcessors)` with a list of `Processor` objects. See **Chapter 11 of Volume I** for an example of annotation processing.

The `getTask` method returns the task object but does not yet start the compilation process. The `CompilationTask` class extends `Callable<Boolean>`. You can submit it to an `ExecutorService` for concurrent execution, or you can just make a synchronous call:

```
Boolean result = task.call();
```

8.1.3. Capturing Diagnostics

To listen to error messages, install a `DiagnosticListener`. The listener receives a `Diagnostic` object whenever the compiler reports a warning or error message. The `DiagnosticCollector` class implements this interface. It simply collects all diagnostics so that you can iterate through them after the compilation is complete.

```
var collector = new DiagnosticCollector<JavaFileObject>();
compiler.getTask(null, fileManager, collector, null, null, sources).call();
for (Diagnostic<? extends JavaFileObject> d : collector.getDiagnostics())  {
    IO.println(d);
}
```

A `Diagnostic` object contains information about the problem location (including file name, line number, and column number) as well as a human-readable description.

You can also install a `DiagnosticListener` to the standard file manager, in case you want to trap messages about missing files:

```
StandardJavaFileManager stdFileManager
    = compiler.getStandardFileManager(diagnosticsListener, null, null);
```

8.1.4. Reading Source Files from Memory

If you generate source code on the fly, you can have it compiled from memory, without having to save files to disk. Use this class to hold the code:

```
public class StringSource extends SimpleJavaFileObject {
    private String code;

    StringSource(String name, String code) {
        super(URI.create("string:///" + name.replace(".", "/") + ".java"), Kind.SOURCE);
        this.code = code;
    }

    public CharSequence getCharContent(boolean ignoreEncodingErrors) {
        return code;
    }
}
```

Then generate the code for your classes and give the compiler a list of `StringSource` objects:

```
List<StringSource> sources = List.of(
    new StringSource(className1, class1CodeString),
    new StringSource(className2, class2CodeString),
    . . .);
task = compiler.getTask(null, fileManager, diagnosticsListener, null, null, sources);
```

8.1.5. Writing Byte Codes to Memory

If you compile classes on the fly, there is no need to save the class files to disk. You can save them to memory and load them right away.

First, here is a class for holding the bytes:

```
public class ByteArrayClass extends SimpleJavaFileObject {
    private ByteArrayOutputStream out;

    ByteArrayClass(String name) {
        super(URI.create("bytes:///" + name.replace(".", "/") + ".class"), Kind.CLASS);
    }

    public byte[] getCode()  {
        return out.toByteArray();
    }

    public OutputStream openOutputStream() throws IOException {
        out = new ByteArrayOutputStream();
```

```
        return out;
    }
}
```

Next, you need to configure the file manager to use these classes for output:

```
var classes = new ArrayList<ByteArrayClass>();
StandardJavaFileManager stdFileManager
    = compiler.getStandardFileManager(null, null, null);
JavaFileManager fileManager
    = new ForwardingJavaFileManager<>(stdFileManager) {
        public JavaFileObject getJavaFileForOutput(Location location,
                String className, Kind kind, FileObject sibling)
                throws IOException {
            if (kind == Kind.CLASS) {
                 ByteArrayClass outfile = new ByteArrayClass(className);
                 classes.add(outfile);
                 return outfile;
            }
            else
                 return super.getJavaFileForOutput(location, className, kind, sibling);
        }
    };
```

To load the classes, you need a class loader (see **Chapter 9**):

```
public class ByteArrayClassLoader extends ClassLoader {
    private Iterable<ByteArrayClass> classes;

    public ByteArrayClassLoader(Iterable<ByteArrayClass> classes) {
        this.classes = classes;
    }

    public Class<?> findClass(String name) throws ClassNotFoundException {
        for (ByteArrayClass cl : classes) {
            if (cl.getName().equals("/" + name.replace(".", "/") + ".class")) {
                byte[] bytes = cl.getCode();
                return defineClass(name, bytes, 0, bytes.length);
            }
        }
        throw new ClassNotFoundException(name);
    }
}
```

After compilation has finished, call the `Class.forName` method with that class loader:

```
ByteArrayClassLoader loader = new ByteArrayClassLoader(classes);
Class<?> cl = Class.forName(className, true, loader);
```

8.1.6. An Example: Dynamic Java Code Generation

In the Jakarta Server Pages (JSP) technology for dynamic web pages, you can mix HTML with snippets of Java code, for example:

```
<p>The current date and time is <b><%= LocalDateTime.now() %></b>.</p>
```

The JSP engine dynamically compiles the Java code into a servlet. In our sample application, we use a simpler example and generate code for a very simple worksheet format. Authors include text, Java expressions (enclosed in backticks), and Java statements (enclosed in triple backticks) into a worksheet. A `???` denotes a line of input. The worksheet is compiled into a Java class that prints the text and the values of the expressions, and executes the Java statements. Here is a sample worksheet:

```
What is your name?
```
String name = ???;
```
How old are you?
```
int age = Integer.parseInt(???);
```
Hello, `name`! Next year, you will be `age + 1`.
```

This is a very minimal text version of a programming notebook (**https://docs.jupyter.org/en/latest/#what-is-a-notebook**). The sheet is translated into a class whose `main` method has print statements to emit the text and expression values. The statements are included verbatim. One can envision an enhancement of this program that displays images and tables, just like a real programming notebook.

The `buildSource` method in the program of **Listing 8.1** builds up this code and places it into a `StringSource` object. That object is passed to the Java compiler.

As described in the preceding section, we use a `ForwardingJavaFileManager` that constructs a `ByteArrayClass` object for every compiled class. These objects capture the class file that is generated when the class is compiled. After compilation, we use the class loader from the preceding section to load the class and execute its `main` method.

This example demonstrates how you can use dynamic compilation with in-memory source and class files.

Listing 8.1 v2ch08/compiler/CompilerDemo.java

```
1  package v2ch08.compiler;
2
3  import module java.base;
4  import module java.compiler;
5
6  import javax.tools.JavaFileObject.Kind;
7  import javax.tools.ToolProvider;
8
9  /**
10  * This program demonstrates compiling Java code in a Java program.
```

```
 */
public class CompilerDemo {
    void main(String[] args) throws Exception {
        JavaCompiler compiler = ToolProvider.getSystemJavaCompiler();

        var classFileObjects = new ArrayList<ByteArrayClass>();

        var diagnosticsListener = new DiagnosticCollector<JavaFileObject>();

        JavaFileManager fileManager
            = compiler.getStandardFileManager(diagnosticsListener, null, null);
        fileManager = new ForwardingJavaFileManager<>(fileManager) {
            public JavaFileObject getJavaFileForOutput(Location location, String className,
                    Kind kind, FileObject sibling) throws IOException {
                if (kind == Kind.CLASS) {
                    var fileObject = new ByteArrayClass(className);
                    classFileObjects.add(fileObject);
                    return fileObject;
                }
                else
                    return super.getJavaFileForOutput(location, className, kind, sibling);
            }
        };

        String mainClassName = "v2ch08.compiler.Main";
        String worksheetName = args.length > 0 ? args[0] : "v2ch08/compiler/worksheet";

        StandardJavaFileManager stdFileManager = compiler.getStandardFileManager(null, null,
                null);
        var sources = new ArrayList<JavaFileObject>();
        for (JavaFileObject file : stdFileManager.getJavaFileObjectsFromStrings(
                List.of(mainClassName.replace(".", "/") + ".java")))
            sources.add(file);

        JavaFileObject source = buildSource(mainClassName, Path.of(worksheetName));
        JavaCompiler.CompilationTask task = compiler.getTask(null, fileManager,
                diagnosticsListener, null, null, List.of(source));
        Boolean result = task.call();

        for (Diagnostic<? extends JavaFileObject> d : diagnosticsListener.getDiagnostics())
            IO.println(d.getKind() + ": " + d.getMessage(null));
        fileManager.close();
        if (!result) {
            IO.println("Compilation failed.");
            System.exit(1);
        }

        var loader = new ByteArrayClassLoader(classFileObjects);
        Class<?> mainClass = loader.loadClass(mainClassName);
        mainClass.getMethod("main").invoke(null);
    }

    /**
     * Builds the source for the class that implements the worksheet actions.
     * @param className the name of the class to be built
     * @param worksheet the file containing the worksheet instructions
     * @return a file object containing the source in a string builder
```

```
     */
    JavaFileObject buildSource(String className, Path worksheet) throws IOException {
        var builder = new StringBuilder();
        int n = className.lastIndexOf(".");
        if (n >= 0) {
            String packageName = className.substring(0, n);
            className = className.substring(n + 1);
            builder.append("package ").append(packageName).append(";\n\n");
        }
        builder.append("public class ").append(className).append(" {\n");
        builder.append("public static void main() {\n");
        boolean verbatim = false;
        for (String line : Files.readAllLines(worksheet)) {
            if (line.equals("```"))
                verbatim = !verbatim;
            else if (verbatim)
                builder.append(line.replace("???", "IO.readln()")).append("\n");
            else {
                String[] fragments = line.split("`");
                for (int i = 0; i < fragments.length; i++) {
                    builder.append("IO.print(");
                    if (i % 2 == 0)
                        builder.append("\"").append(
                                fragments[i].replace("\\", "\\\\").replace("\"", "\\\""))
                                .append("\"");
                    else
                        builder.append(fragments[i]);
                    builder.append(");\n");
                }
                builder.append("IO.println()\n;");
            }
        }

        builder.append("} }\n");
        return new StringSource(className, builder.toString());
    }
}
```

javax.tools.Tool **6**

- `int run(InputStream in, OutputStream out, OutputStream err, String... arguments)` runs the tool with the given input, output, and error streams and the given arguments. Returns `0` for success, a nonzero value for failure.

javax.tools.JavaCompiler **6**

- `StandardJavaFileManager getStandardFileManager(DiagnosticListener<? super JavaFileObject> diagnosticListener, Locale locale, Charset charset)` gets the standard file manager for this compiler. You can supply `null` for default error reporting, locale, and character set.
- `JavaCompiler.CompilationTask getTask(Writer out, JavaFileManager fileManager, DiagnosticListener<? super JavaFileObject> diagnosticListener, Iterable<String>`

`options, Iterable<String> classesForAnnotationProcessing, Iterable<? extends JavaFileObject> sourceFiles)`
gets a compilation task that, when called, will compile the given source files. See the discussion in the preceding section for details.

javax.tools.StandardJavaFileManager **6**

- `Iterable<? extends JavaFileObject> getJavaFileObjectsFromStrings(Iterable<String> fileNames)`
- `Iterable<? extends JavaFileObject> getJavaFileObjectsFromPaths(Collection<? extends Path> paths)` **13**
- `Iterable<? extends JavaFileObject> getJavaFileObjectsFromFiles(Iterable<? extends File> files)`
  translate a sequence of file names or files into a sequence of `JavaFileObject` instances.

javax.tools.JavaCompiler.CompilationTask **6**

- `Boolean call()`
  performs the compilation task.

`javax.tools.DiagnosticCollector<S>` **6**

- `DiagnosticCollector()`
  constructs an empty collector.
- `List<Diagnostic<? extends S>> getDiagnostics()`
  gets the collected diagnostics.

javax.tools.Diagnostic<S> **6**

- `S getSource()`
  gets the source object associated with this diagnostic.
- `Diagnostic.Kind getKind()`
  gets the type of this diagnostic—one of `ERROR`, `WARNING`, `MANDATORY_WARNING`, `NOTE`, or `OTHER`.
- `String getMessage(Locale locale)`
  gets the message describing the issue raised in this diagnostic. Pass `null` for the default locale.
- `long getLineNumber()`
- `long getColumnNumber()`
  get the position of the issue raised in this diagnostic.

`javax.tools.SimpleJavaFileObject` **6**

- `CharSequence getCharContent(boolean ignoreEncodingErrors)`
  override this method for a file object that represents a source file and produces the source code.
- `OutputStream openOutputStream()`
  override this method for a file object that represents a class file and produces a stream to which the bytecodes can be written.

`javax.tools.ForwardingJavaFileManager<M extends JavaFileManager>` 6

- `protected ForwardingJavaFileManager(M fileManager)`
  constructs a `JavaFileManager` that delegates all calls to the given file manager.
- `FileObject getJavaFileForOutput(JavaFileManager.Location location, String className, JavaFileObject.Kind kind, FileObject sibling)`
  Intercept this call if you want to substitute a file object for writing class files; `kind` is one of `SOURCE`, `CLASS`, `HTML`, or `OTHER`.

8.2. Scripting for the Java Platform

A scripting language is a language that avoids the usual edit/compile/link/run cycle by interpreting the program text at runtime. Scripting languages have a number of advantages:

- Rapid turnaround, encouraging experimentation
- Changing the behavior of a running program
- Enabling customization by program users

On the other hand, most scripting languages lack features that are beneficial for programming complex applications, such as strong typing, encapsulation, and modularity.

It is therefore tempting to combine the advantages of scripting and traditional languages. The scripting API lets you do just that for the Java platform. It enables you to invoke scripts written in JavaScript, Groovy, Ruby, and even exotic languages such as Scheme and Haskell, from a Java program. For example, the Renjin project (**https://www.renjin.org**) provides a Java implementation of the R programming language, which is commonly used for statistical programming, together with an "engine" of the scripting API.

When looking for a compatible implementation of your favorite language, search for JSR 223 support. (Java Specification Request 223 developed the scripting API before it was integrated into Java.)

The JMeter performance analysis tool (**https://jmeter.apache.org/**) is an example of a program that allows users to write scripts in any language with Java scripting support. The Maven build tool has a plugin (**https://maven.apache.org/plugins/maven-scripting-plugin/index.html**) for writing build steps in any language that has a Java scripting engine.

In the following sections, I'll show you how to select an engine for a particular language, how to execute scripts, and how to make use of advanced features that some scripting engines offer.

8.2.1. Getting a Scripting Engine

A scripting engine is a library that can execute scripts in a particular language. When the virtual machine starts, it discovers the available scripting engines. To enumerate them, construct a `ScriptEngineManager` and invoke the `getEngineFactories` method. You can ask each engine factory for the supported engine names, MIME types, and file extensions. **Table 8.1** shows typical values.

The examples in this chapter use the Rhino scripting engine, available at **https://github.com/mozilla/rhino**, that implements an ancient version of JavaScript.

Table 8.1: Properties of Scripting Engine Factories

Engine	Names	MIME Types	Extensions
Rhino (JavaScript)	rhino, Rhino, JavaScript, javascript	`application/javascript`, `application/ecmascript`, `text/javascript`, `text/ecmascript`	`js`
Groovy	groovy	None	`groovy`
Renjin	Renjin	`text/x-R`	`R`, `r`, `S`, `s`

Usually, you know which engine you need, and you can simply request it by name, MIME type, or extension. For example:

```
var manager = new ScriptEngineManager();
ScriptEngine engine = manager.getEngineByName("rhino");
```

You need to provide the JAR files that implement the script engine on the classpath.



Note: The Oracle JDK used to contain a JavaScript engine called Nashorn, but it has been removed in Java 15. It is available as a standalone project at **https://github.com/openjdk/nashorn**.

`javax.script.ScriptEngineManager` 6

- `List<ScriptEngineFactory> getEngineFactories()`
  gets a list of all discovered engine factories.
- `ScriptEngine getEngineByName(String name)`
- `ScriptEngine getEngineByExtension(String extension)`
- `ScriptEngine getEngineByMimeType(String mimeType)`
  get the script engine with the given name, script file extension, or MIME type.

javax.script.ScriptEngineFactory 6

- `List<String> getNames()`
- `List<String> getExtensions()`
- `List<String> getMimeTypes()`
  get the names, script file extensions, and MIME types under which this factory is known.

8.2.2. Script Evaluation and Bindings

Once you have an engine, you can call a script simply by invoking

```
Object result = engine.eval(scriptString);
```

If the script is stored in a file, open a `Reader` and call

```
Object result = engine.eval(reader);
```

You can invoke multiple scripts on the same engine. If one script defines variables, functions, or classes, most scripting engines retain the definitions for later use. For example,

```
engine.eval("n = 1728");
Object result = engine.eval("n + 1");
```

will return `1729`.



Note: To find out whether it is safe to concurrently execute scripts in multiple threads, call

```
Object param = factory.getParameter("THREADING");
```

The returned value is one of the following:

- `null`: Concurrent execution is not safe.
- `"MULTITHREADED"`: Concurrent execution is safe. Effects from one thread might be visible from another thread.
- `"THREAD-ISOLATED"`: In addition to `"MULTITHREADED"`, different variable bindings are maintained for each thread.
- `"STATELESS"`: In addition to `"THREAD-ISOLATED"`, scripts do not alter variable bindings.

You will often want to add variable bindings to the engine. A binding consists of a name and an associated Java object. For example, consider these statements:

```
engine.put("k", 1728);
Object result = engine.eval("k + 1");
```

The script code reads the definition of `k` from the bindings in the "engine scope." This is particularly important because most scripting languages can access Java objects, often with a syntax that is simpler than the Java syntax. For example,

```
engine.put("d", LocalDate.now());
Object result = engine.eval("d.month"); // calls getMonth
```

Conversely, you can retrieve variables that were bound by scripting statements:

```
engine.eval("n = 1728");
Object result = engine.get("n");
```

In addition to the engine scope, there is also a global scope. Any bindings that you add to the `ScriptEngineManager` are visible to all engines.

Instead of adding bindings to the engine or global scope, you can collect them in an object of type `Bindings` and pass it to the `eval` method:

```
Bindings scope = engine.createBindings();
scope.put("d", LocalDate.now());
engine.eval(scriptString, scope);
```

This is useful if a set of bindings should not persist for future calls to the `eval` method.



Note: You might want to have scopes other than the engine and global scopes. For example, a web container might need request and session scopes. However, then you are on your own. You will need to write a class that implements the `ScriptContext` interface, managing a collection of scopes. Each scope is identified by an integer number, and scopes with lower numbers should be searched first. (The standard library provides a `SimpleScriptContext` class, but it only holds global and engine scopes.)

javax.script.ScriptEngine 6

- `Object eval(String script)`
- `Object eval(Reader reader)`
- `Object eval(String script, Bindings bindings)`
- `Object eval(Reader reader, Bindings bindings)`
  evaluate the script given by the string or reader, subject to the given bindings.
- `Object get(String key)`
- `void put(String key, Object value)`
  get or put a binding in the engine scope.
- `Bindings createBindings()`
  creates an empty `Bindings` object suitable for this engine.

`javax.script.ScriptEngineManager` 6

- `Object get(String key)`
- `void put(String key, Object value)`
  get or put a binding in the global scope.

javax.script.Bindings 6

- `Object get(String key)`
- `void put(String key, Object value)`
  get or put a binding into the scope represented by this `Bindings` object.

8.2.3. Redirecting Input and Output

You can redirect the standard input and output of a script by calling the `setReader` and `setWriter` methods of the script context. For example,

```
var writer = new StringWriter();
engine.getContext().setWriter(new PrintWriter(writer, true));
```

Any output written with the JavaScript `print` or `println` functions is sent to `writer`.

The `setReader` and `setWriter` methods only affect the scripting engine's standard input and output sources. For example, if you execute the JavaScript code

```
println("Hello");
java.lang.System.out.println("World");
```

only the first output is redirected.

The Rhino engine does not have the notion of a standard input source. Calling `setReader` has no effect.

javax.script.ScriptEngine 6

- `ScriptContext getContext()`
  gets the default script context for this engine.

javax.script.ScriptContext 6

- `Reader getReader()`
- `void setReader(Reader reader)`
- `Writer getWriter()`
- `void setWriter(Writer writer)`
- `Writer getErrorWriter()`
- `void setErrorWriter(Writer writer)`
  get or set the reader for input or writer for normal or error output.

8.2.4. Calling Scripting Functions and Methods

With some script engines, you can invoke a scripting language function directly in your Java code, without having to evaluate a script code snippet that makes the call. This is useful if you allow users to implement a service in a scripting language of their choice, so that you can call it from Java.

The script engines that offer this functionality implement the `Invocable` interface. In particular, the Rhino engine implements `Invocable`.

To call a function, call the `invokeFunction` method with the function name, followed by the function arguments:

```
// Define greet function in JavaScript
engine.eval("function greet(how, whom) { return how + ', ' + whom + '!' }");

// Call the function with arguments "Hello", "World"
result = ((Invocable) engine).invokeFunction("greet", "Hello", "World");
```

If the scripting language is object-oriented, call `invokeMethod`:

```
// Define Greeter class in JavaScript
engine.eval("""
function Greeter(how) { this.how = how }
Greeter.prototype.welcome =
    function (whom) { return this.how + ', ' + whom + '!' }
""");

// Construct an instance
Object yo = engine.eval("new Greeter('Yo')");

// Call the welcome method on the instance
result = ((Invocable) engine).invokeMethod(yo, "World");
```



Note: Rhino does not support the modern JavaScript class syntax. For more information on how to define classes in JavaScript the old-fashioned way, see *JavaScript—The Good Parts* by Douglas Crockford (O'Reilly, 2008).



Note: If the script engine does not implement the `Invocable` interface, you might still be able to call a method in a language-independent way. The `getMethodCallSyntax` method of the `ScriptEngineFactory` interface produces a string that you can pass to the `eval` method. For this, however, all method parameters must be bound to names, whereas `invokeMethod` can be called with arbitrary values.

You can go a step further and ask the scripting engine to implement a Java interface. Then you can call scripting functions and methods with the Java method call syntax.

The details depend on the scripting engine, but typically you need to supply a function for each method of the interface. For example, consider a Java interface

```
public interface Greeter {
    String welcome(String whom);
}
```

If you define a global function with the same name in Rhino, you can call it through this interface:

```
// Define welcome function in JavaScript
engine.eval("function welcome(whom) { return 'Hello, ' + whom + '!' }");

// Get a Java object and call a Java method
Greeter g = ((Invocable) engine).getInterface(Greeter.class);
result = g.welcome("World");
```

In an object-oriented scripting language, you can access a script class through a matching Java interface. For example, here is how to call an object of the JavaScript `Greeter` class with Java syntax:

```
Greeter g = ((Invocable) engine).getInterface(yo, Greeter.class);
result = g.welcome("World");
```

In summary, the `Invocable` interface is useful if you want to call scripting code from Java without worrying about the scripting language syntax.

javax.script.Invocable 6

- `Object invokeFunction(String name, Object... args)`
- `Object invokeMethod(Object thiz, String name, Object... args)`
  invoke the function or method with the given name, passing the given arguments.
- `<T> T getInterface(Class<T> clasz)`
  returns an implementation of the given interface, implementing the methods with functions in the scripting engine.
- `<T> T getInterface(Object thiz, Class<T> clasz)`
  returns an implementation of the given interface, implementing the methods with the methods of the given object.

8.2.5. Compiling a Script

Some scripting engines can compile scripting code into an intermediate form for efficient execution. Those engines implement the `Compilable` interface. The following example shows how to compile and evaluate code contained in a script file:

```
if (engine instanceof Compilable compilableEngine) {
    Reader reader = Files.newBufferedReader(path);
    CompiledScript script = compilableEngine.compile(reader);
    script.eval();
}
```

Of course, it only makes sense to compile a script if you need to execute it repeatedly.

javax.script.Compilable 6

- `CompiledScript compile(String script)`
- `CompiledScript compile(Reader reader)`
  compile the script given by a string or reader.

`javax.script.CompiledScript` 6

- `Object eval()`
- `Object eval(Bindings bindings)`
  evaluate this script.

8.2.6. An Example: Script Sheets

Have a look at the program in **Listing 8.2** that executes a sheet in the same format as the example in **Section 8.1.6**. However, this time, the code is in a scripting language, and it is evaluated when the program runs.

The classpath must contain a scripting engine. Download Rhino and the Rhino engine JAR from **https://mvnrepository.com/artifact/org.mozilla/rhino** and

https://mvnrepository.com/artifact/org.mozilla/rhino-engine and place them into a directory `jars`. Start the program like this:

```
java -classpath .:v2ch08/script/rhino-1.7.14.jar:v2ch08/script/rhino-engine-1.7.14.jar \
    v2ch08/script/ScriptDemo.java rhino v2ch08/script/worksheet
```

Listing 8.2 v2ch08/script/ScriptDemo.java

```
1  package v2ch08.script;
2
3  import module java.base;
4  import module java.scripting;
5
6  /**
7   * This program demonstrates the Java scripting engine.
8   */
9  class ScriptDemo {
10     void main(String[] args) throws Exception {
11         var manager = new ScriptEngineManager();
12         String language;
13         if (args.length == 0) {
14             IO.println("Available factories: ");
15             for (ScriptEngineFactory factory : manager.getEngineFactories())
16                 IO.println(factory.getEngineName());
17             return;
18         }
19         else
20             language = args[0];
21
22         ScriptEngine engine = manager.getEngineByName(language);
23         if (engine == null) {
24             System.err.println("No engine for " + language);
25             System.exit(1);
26         }
27
28         String sheetName = args[1];
29         boolean verbatim = false;
30         var verbatimCode = new StringBuilder();
31         for (String line : Files.readAllLines(Path.of(sheetName))) {
32             if (line.equals("```")) {
33                 verbatim = !verbatim;
34                 if (!verbatim) {
35                     engine.eval(verbatimCode.toString());
36                     verbatimCode.delete(0, verbatimCode.length());
37                 }
38             }
39             else if (verbatim)
40                 verbatimCode.append(line).append("\n");
41             else {
42                 String[] fragments = line.split("`");
43                 for (int i = 0; i < fragments.length; i++) {
44                     IO.print(i % 2 == 0 ? fragments[i] : engine.eval(fragments[i]));
45                 }
46                 IO.println();
```

```
            }
        }
    }
}
```

Listing 8.3 v2ch08/script/worksheet

````
```
var name = java.lang.IO.readln("What is your name? ");
var age = 1 * java.lang.IO.readln("How old are you? ");
```
Hello, `name`! Next year, you will be `age + 1`.
````

Previous chapter
< 7. Internationalization

Next chapter
9. Security >